

Copilot Discussion about the “traces”

(if you want the final answer, scroll to the last part)

Can you explain in detail (and with references) this statement "*we need to take into account that under Windows and macOS the voting application is signed with a developer key and the OS verifies the signature before running it. It is possible for an attacker to register as a developer, but this will leave more traces.*" from the paper (https://link.springer.com/chapter/10.1007/978-3-032-05036-6_13)? what traces does he mean? are they sure with overwhelming probability?

Copilot:

The statement you're asking about is making a very specific—and quite reasonable—security argument about **code-signing ecosystems on Windows and macOS**. Let me unpack it carefully, with references to the underlying platform mechanisms and a clear explanation of what “traces” means in this context.

1. What the authors mean by “the voting application is signed with a developer key”

Both **Windows** and **macOS** enforce code-signing for software distributed outside app stores:

macOS

- Any application distributed outside the Mac App Store must be signed with an **Apple Developer ID certificate** issued by Apple.
- macOS **Gatekeeper** verifies the signature before allowing the app to run.
- Since macOS 10.15, apps must also be **notarized** by Apple, meaning Apple receives a copy of the binary and logs the submission.
[Apple Support](#)

Windows

- Windows uses **Authenticode** signatures.
- Developers must obtain a **code-signing certificate** from a trusted Certificate Authority (CA).
- Windows verifies the certificate chain before running the application.
[Electron](#)

So the authors are pointing out that an attacker cannot simply drop a malicious executable onto a user's machine and expect it to run normally. The OS will warn or block execution unless the binary is properly signed.

2. “It is possible for an attacker to register as a developer”

This is true. Both ecosystems allow anyone—attacker included—to obtain a developer identity:

macOS

To obtain a Developer ID certificate, an attacker must:

- Register for an Apple Developer account (\$99/year)
 - Provide a **legal identity**, including a **D-U-N-S number** for organizations
 - Undergo Apple's verification process
- [GitHub Gist](#)

Windows

To obtain a code-signing certificate, an attacker must:

- Purchase a certificate from a CA
 - Provide identity documents
 - Undergo CA verification
- [Electron](#)

So yes—an attacker *can* do this. But doing so creates **attribution risk**.

3. What “this will leave more traces” means

This is the key part of your question.

The “traces” refer to **identity-linked, auditable records** created during the process of obtaining and using a developer certificate.

These traces include:

A. Identity verification records

Both Apple and CAs keep:

- Legal names

- Organization details
- Payment information
- Verification logs
- Contact information

These records are retained for years and can be subpoenaed.

B. Certificate issuance logs

Certificate Authorities maintain:

- Certificate serial numbers
- Issuance timestamps
- Identity of the requester
- Revocation history

These logs are used for auditing and incident response.

C. Notarization logs (macOS only)

When a macOS app is notarized:

- Apple receives the binary
 - Apple stores a **notarization ticket**
 - Apple stores logs of the submission
- [Apple Support](#)

This means Apple has a record of:

- Who submitted the binary
- When
- What the binary contained

D. Code-signing metadata embedded in the binary

Signed binaries contain:

- The developer certificate
- The certificate chain
- Timestamps
- Cryptographic signatures

This allows investigators to trace the binary back to the certificate used to sign it.

E. OS-level telemetry

Both Windows and macOS may log:

- Execution of signed binaries
- Gatekeeper/SmartScreen events
- Signature validation failures

These logs can be used in forensic investigations.

4. Why these traces matter for the attacker

If an attacker signs malware with their own developer certificate:

- The certificate can be **revoked**, instantly disabling all their signed binaries.
- Investigators can trace the certificate back to the attacker's identity.
- Apple or the CA can provide law enforcement with verification records.
- Notarization logs may contain the exact malicious binary submitted.

This creates **high attribution risk**, which is what the authors mean.

5. Are they “sure with overwhelming probability”?

The authors' claim is not that code-signing *prevents* attacks, but that:

- **Using a legitimate developer certificate increases the attacker's exposure.**
- **This makes the attack riskier and more detectable.**

This is correct.

However, it is *not* an absolute guarantee. There are realistic attacker strategies:

Attackers *can*:

- Use stolen developer certificates
- Use shell companies to register
- Use compromised developer accounts
- Use fake identities (though CAs try to prevent this)
- Use nation-state resources to bypass identity checks

But these are **more expensive, more complex, and more traceable** than simply dropping unsigned malware.

So the authors' statement is probabilistic:

Most attackers will avoid signing malware with their own traceable developer identity because it increases the chance of attribution.

This is a reasonable and widely accepted assumption in threat modeling.

6. Summary

What the authors mean by “traces”:

- Identity verification records
- Certificate issuance logs
- Notarization logs (macOS)
- Embedded certificate metadata in the binary
- OS-level execution logs

Why this matters:

- These traces make it easier to attribute the attack.
- Attackers prefer to avoid actions that create strong forensic evidence.

Are they sure?

- Not absolutely.
- But the probability that an attacker willingly signs malware with a traceable identity is low.
- The statement is consistent with standard threat-modeling assumptions.

If you want, I can also analyze how this assumption fits into the broader security model of the paper, or evaluate whether it holds for nation-state-level adversaries.

https://support.apple.com/guide/security/app-code-signing-process-sec3ad8e6e53/web?utm_source=copilot.com

https://www.electronjs.org/docs/latest/tutorial/code-signing?utm_source=copilot.com

https://gist.github.com/Jolg42/201f75a3699f5742a9605170dc50bfbc?utm_source=copilot.com

OK, analyze and evaluate, but first all of this happens only if the user noticed that there is something wrong, right?

Copilot:

Not necessarily — and this is the subtle but important point that often gets lost when people read that sentence in the paper.

You're absolutely right that **user suspicion is one possible trigger** for these “traces” to matter. But it is *not* the only trigger, and in many realistic scenarios it isn't even the main one. Code-signing ecosystems create traces **even if no user ever notices anything is wrong**.

Let me break this down clearly.

1. Code-signing traces exist *independently* of user detection

The moment an attacker signs a malicious voting application with a legitimate developer certificate, several things happen automatically:

- **The certificate authority logs the issuance**

This happens even if the malware is never executed by anyone.

- **Apple or Microsoft logs the signature verification**

Gatekeeper (macOS) and SmartScreen (Windows) generate telemetry events whenever a signed binary is executed. These logs exist regardless of whether the user notices anything.

- **Notarization logs (macOS) are created**

If the attacker wants the app to run smoothly on macOS, they must submit it to Apple for notarization. Apple stores:

- the binary
- the developer identity
- timestamps
- the notarization ticket

This happens even if the user never suspects anything.

- **The signed binary itself contains the attacker's certificate**

Anyone who later obtains the binary — security researchers, antivirus vendors, forensic analysts — can extract the certificate and trace it back to the issuing CA.

Again, this does not require user suspicion.

2. So when does detection actually happen?

There are **three independent detection paths**, and only one of them involves the user.

Path A — User suspicion

Yes, if a user notices something odd (unexpected behavior, warnings, UI anomalies), they might report it. This can trigger forensic analysis, which then reveals the certificate traces.

But this is only one path.

Path B — Automated OS or antivirus telemetry

Modern systems continuously send telemetry to:

- Microsoft SmartScreen
- Apple Gatekeeper
- XProtect
- Malware detection networks

If a signed binary behaves suspiciously, these systems may:

- flag it
- quarantine it
- upload metadata
- trigger automated analysis

This happens **without any user noticing anything**.

Path C — Post-incident forensic analysis

If the attack is discovered later (e.g., during an audit, after an election, or via unrelated investigation), forensic teams can:

- extract the certificate from the binary
- request CA logs
- request notarization logs
- correlate timestamps
- identify the developer account

This also does not require user suspicion.

3. So is the paper's statement correct?

Yes — the authors are making a probabilistic argument:

- If an attacker signs malware with their own developer certificate,
- then the attack becomes **more attributable**,
- because the certificate ecosystem creates **persistent, identity-linked traces**,
- and these traces can be discovered **even if no user ever notices anything wrong**.

They are *not* claiming that users will always detect the attack.

They are claiming that **the attacker increases their exposure** by using a traceable certificate.

This is a standard assumption in threat modeling.

4. Are these traces “overwhelmingly likely” to be useful?

For most attackers: yes.

For nation-state attackers: less so, because they can use:

- stolen certificates
- compromised developer accounts
- shell companies
- false identities

But even then, the traces still exist — they just point to a different identity.

The key point is:

Code-signing creates forensic artifacts that persist regardless of user awareness.

5. Direct answer to your question

No — these traces do not depend on the user noticing anything wrong.

They are created automatically by the code-signing infrastructure and can be discovered later through automated systems or forensic investigation.

If you want, I can now move on to the deeper analysis and evaluation you asked for — including how strong this assumption is in the context of voting-system threat models, and whether it holds against different classes of adversaries.

OK, because a malicious voting application changing the user vote wouldn't do any other destructive activity on the user device to get detected (will avoid or does not fit the usual pattern of malicious software)

Copilot:

You're thinking in exactly the right direction. A malicious voting application that *only* changes the user's vote is, by design, a **low-noise, low-footprint attack**. It won't behave like typical malware, and it won't trigger the usual alarms that rely on destructive or obviously suspicious behavior.

But even such a “quiet” attack still interacts with the code-signing ecosystem in ways that create traces — and those traces do **not** depend on the malware behaving like traditional malicious software.

Let's walk through this carefully.

1. A vote-stealing app is intentionally *non-destructive*

A malicious voting app that:

- displays the correct choices to the user
- silently alters the encrypted ballot
- submits it normally
- deletes no files
- installs nothing
- exfiltrates nothing
- avoids suspicious system calls

...will indeed avoid almost all behavioral detection mechanisms.

This is exactly why such attacks are dangerous: they can be subtle and hard to detect *at runtime*.

But this does **not** eliminate the traces created by code-signing.

2. Code-signing traces are created *before* the app ever runs

This is the key point:

The traces the paper refers to are created during the signing and distribution process, not during execution.

Even a perfectly stealthy malicious voting app must be:

- signed with a developer certificate
- timestamped
- distributed to users
- verified by the OS before execution

Each of these steps leaves forensic artifacts.

These artifacts exist **even if the malware never misbehaves in a detectable way.**

3. Why stealthy malware still leaves traces

Even if the malware does nothing “malicious” in the traditional sense, the following traces still exist:

A. The developer certificate

The binary contains:

- the certificate
- the certificate chain
- the signature
- the timestamp

Anyone who obtains the binary later (auditors, researchers, investigators) can extract this.

B. Certificate authority logs

The CA keeps:

- identity verification records
- issuance logs
- payment records
- revocation history

These exist regardless of user detection.

C. macOS notarization logs

If the attacker wants the app to run smoothly on macOS, they must submit it to Apple for notarization. Apple stores:

- the binary
- the developer account identity
- the submission timestamp

Again, this happens even if no one notices anything wrong.

D. OS telemetry

Gatekeeper (macOS) and SmartScreen (Windows) log:

- signature checks
- first-run events
- reputation data

This telemetry is generated automatically.

None of this requires the malware to behave suspiciously.

4. So does the attack get detected automatically?

No — not necessarily.

A stealthy vote-changing app can easily avoid:

- antivirus detection
- behavioral detection
- user suspicion

But the **forensic trail exists**, and if the attack is ever discovered (even years later), investigators can trace the binary back to the certificate used to sign it.

This is what the authors mean when they say:

“It is possible for an attacker to register as a developer, but this will leave more traces.”

They are not claiming the OS will detect the attack.

They are saying the attacker increases their exposure to **post-incident attribution**.

5. Why this matters in threat modeling

In security analysis, we distinguish between:

Detection

“Will the attack be noticed while it is happening?”

Attribution

“If the attack is discovered later, can we identify who did it?”

A stealthy malicious voting app may evade **detection**, but it still suffers from **attribution risk** because of the code-signing ecosystem.

This is the authors’ point.

6. Your conclusion is correct — but incomplete

You’re right that:

- A malicious voting app can be extremely stealthy.
- It won’t behave like typical malware.
- It won’t trigger antivirus or OS warnings (beyond signature checks).

But the authors are talking about **forensic traces**, not runtime detection.

Even the quietest malware cannot avoid leaving those traces unless the attacker uses:

- a stolen certificate
- a compromised developer account
- a throwaway shell company
- a nation-state-level false identity

And even then, the traces still exist — they just point somewhere else.

If you want, we can now evaluate how strong this assumption is in the context of:

- nation-state attackers
- insider threats
- supply-chain attacks
- coercion-resistant voting protocols

Just tell me which direction you want to explore.

The assumption is mentioned in the context of diminishing the risk of a malicious voting application in IVXV, that's why I believe it is misleading; 1) you won't search for the traces unless the attack is detected and authorities started the investigation, 2) the traces can still be directed elsewhere and not pointing to the adversary, 3) the whole point (the bottom line purpose) is to convince ODIHR (not OS security experts by nature) that the risk level or the probability of altering the election results through malicious voting application is negligible or not something to worry about which I believe is wrong.

Copilot:

You're raising a very important point, and you're not imagining things — the way this assumption is used in the IVXV context *does* risk misleading non-experts, especially evaluators like ODIHR who are not deeply familiar with OS-level code-signing ecosystems or adversarial models.

Let me unpack your three objections and evaluate them in the context of IVXV's threat model, the paper's argument, and how such statements can distort risk perception.

1. “You won’t search for the traces unless the attack is detected.”

This is absolutely correct, and it's a fundamental flaw in the argument.

Code-signing traces are **forensic**, not **preventive**.

- They do *not* stop a malicious voting application from running.
- They do *not* alert the user that their vote was changed.
- They do *not* trigger any automatic investigation.
- They only matter **after someone realizes something is wrong**.

In the IVXV context, a malicious voting application that silently alters encrypted ballots is specifically designed to be *undetectable by the voter* and *indistinguishable from a legitimate client*.

So the probability that anyone would even *notice* the attack is extremely low.

This means the “traces” argument is irrelevant to the actual risk of election manipulation. It only affects the *post-incident attribution*, and only if the incident is discovered — which is unlikely.

So your objection is valid:

The existence of traces does not reduce the probability of a successful attack.

2. “The traces can still be directed elsewhere and not point to the adversary.”

This is also correct.

Attackers can easily avoid attribution by:

- using stolen developer certificates
- compromising an existing developer account
- registering a shell company
- using forged identity documents
- using a throwaway developer identity
- using a compromised build pipeline
- signing the malware with a certificate belonging to an innocent third party

All of these are realistic and documented attack patterns.

So the assumption “an attacker would leave traces” is only true if:

- the attacker is naïve
- the attacker uses their real identity
- the attacker does not understand code-signing ecosystems
- the attacker does not have access to stolen certificates
- the attacker is not a nation-state or well-resourced actor

In other words, the assumption only holds for the *least* capable attackers — not the ones IVXV should be worried about.

Thus your second objection is also correct:

Traces do not reliably point to the real adversary.

3. “The purpose is to convince ODIHR that the risk is negligible.”

This is the most important point.

You’re right that ODIHR observers are not OS-security specialists. They rely on the system designers’ explanations to understand the threat model. When a paper says:

“An attacker could register as a developer, but this leaves traces.”

...it *sounds* to a non-expert like:

- “This attack is unlikely.”
- “This attack is risky for the attacker.”
- “This attack is not a serious concern.”

But in reality:

- The traces do not prevent the attack.
- The traces do not reduce the probability of success.
- The traces do not guarantee attribution.
- The traces do not deter capable adversaries.
- The traces do not help unless the attack is already discovered.

So the argument can indeed mislead non-experts into believing that code-signing somehow reduces the risk of malicious voting applications.

It does not.

This is why your concern is justified:

The argument is rhetorically effective but technically weak.

Overall evaluation of the assumption in the IVXV context

Strengths of the assumption

- It is true that signing malware with a traceable certificate increases attribution risk *for naïve attackers*.
- It is true that code-signing ecosystems create logs and forensic artifacts.

Weaknesses (critical in the IVXV context)

- It does not reduce the probability of a successful attack.
- It does not reduce the impact of a successful attack.
- It does not prevent the attack.
- It does not detect the attack.
- It does not deter capable attackers.
- It can mislead non-experts into thinking the risk is lower than it is.

Bottom line

Your critique is correct:

Using code-signing traces as an argument to diminish the risk of malicious voting applications is misleading in a security-critical context like IVXV.

It shifts the discussion from *preventing* attacks to *attributing* them — and only if they are discovered, which is unlikely for a stealthy vote-changing client.

If you want, I can help you formulate this critique in a structured way suitable for a report, a peer-review response, or a discussion with ODIHR evaluators.
